



多线程程序设计

余 烨

本章学习提示

- 本章学习Java多线程的基本思想、方法。
- 学习重点
  - 线程概念的理解
  - 线程的状态
  - Java程序中创建线程的方法
  - 线程常用方法
  - 线程的同步与互斥

目　录

1 线 程 概 述

2 Java线程的状态

3 Java线程的创建

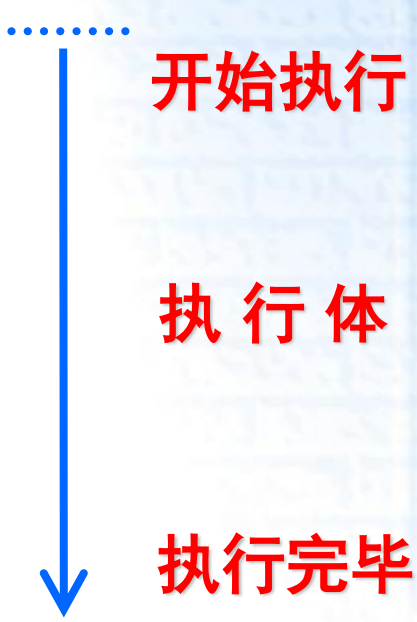
4 多线程的同步和通信

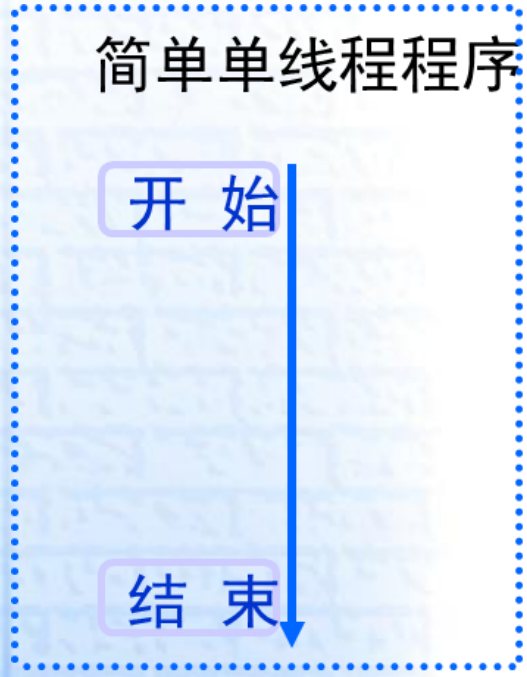
一个例程

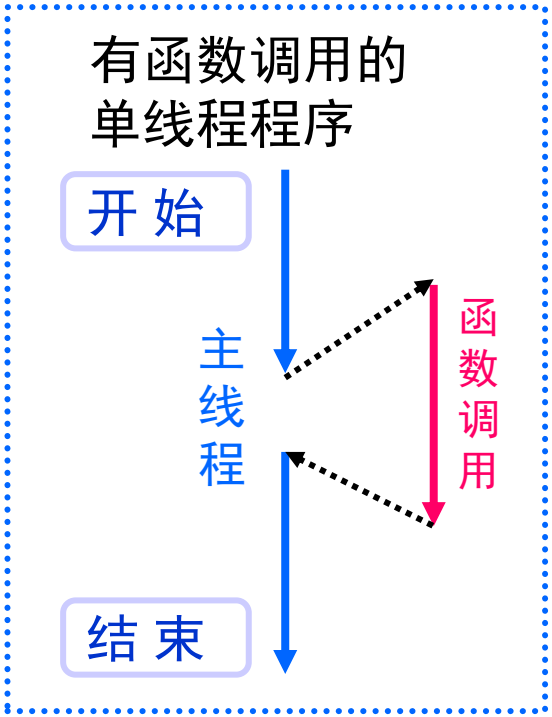
- 在窗体中绘制一个“会运动的、碰壁后会反弹的”小球

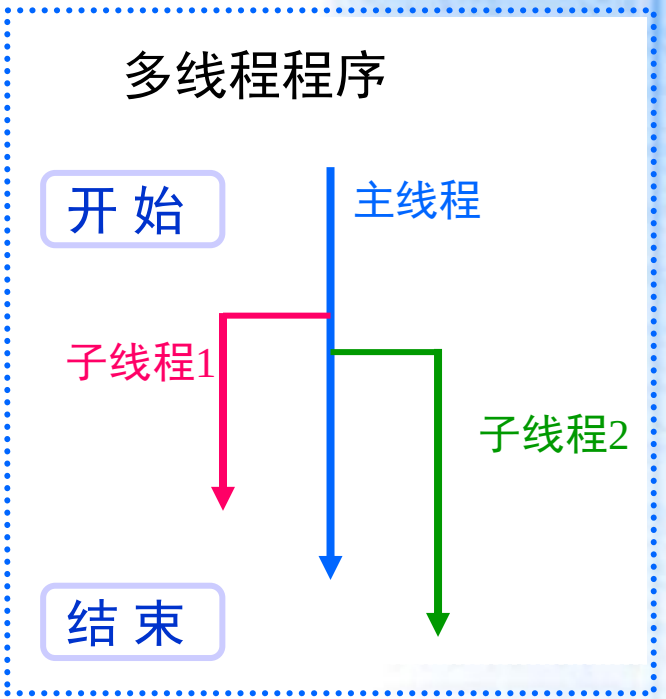
```
// 定义带有“球”的窗体类
class MyFrame {
    …   // 窗体的属性设置
    inner class Ball {
        // 定义球类
       … // 球的属性设置
       … // 球的运动方法
    }
}
```

```
// 程序主类
class DrawSingleBall {
    public static void main(..) {
        // 创建窗口
        new MyFrame
        …
       // 球开始运动
        …
    }
}
```









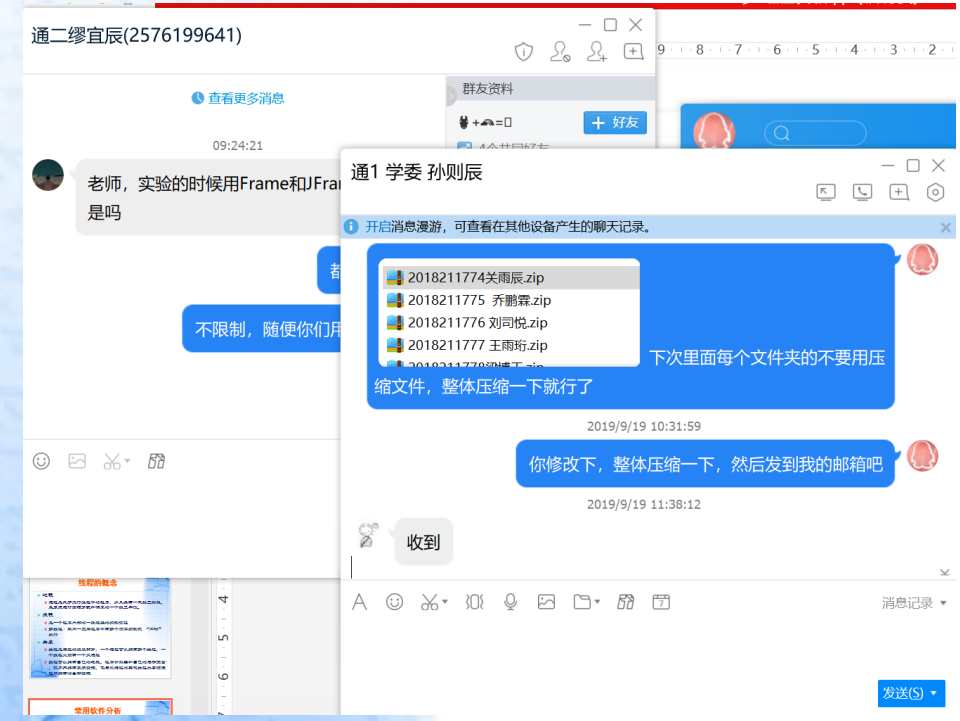
单线程的程序往往功能非常有限！

线程的概念

- 进程
  - 进程是处于运行过程中的程序，并且具有一定独立功能，是系统进行资源分配和调度的一个独立单位。
- 线程
  - 是一个程序内部的一段连续的控制流程
  - 多线程：在同一应用程序中有多个顺序控制流 “同时”执行
- 关系
  - 线程是进程的组成部分，一个进程可以拥有多个线程，一个线程必须有一个父进程
  - 线程可以拥有自己的堆栈、程序计数器和自己的局部变量，但不再拥有系统资源，它与父进程的其他线程共享该进程所拥有的全部资源

常用软件分析

- 目前大多数大型软件都是基于多线程开发的



- 举　例
  - QQ、微信
  - Powerpoint
  - Windows Media Player
  - 暴风影音
  - Winamp
  - 游戏（CS、SC）
  - …

多线程的动机

- ✓ 采用多线程技术的 **动 机**
  - ❖ 问 题： 程序的某部分与特定的事件或资源联系在一起，而设计者不想让这种联系阻碍程序其余部分运行
  - ❖ 解 决：创建一个与事件或资源关联的线程，并且让此线程独立于主线程运行。
- ➢ 一个程序在其执行过程中，可以产生多个线程，形成多条执行线索。每条线程，有产生、存在和消亡的过程。
- ➢ 程序中多个线程，按照自己的执行路线并发工作，独立完成各自的功能，互不干扰。

目　录

1 线 程 概 述

2 Java线程的状态

3 Java线程的创建

4 多线程的同步和通信

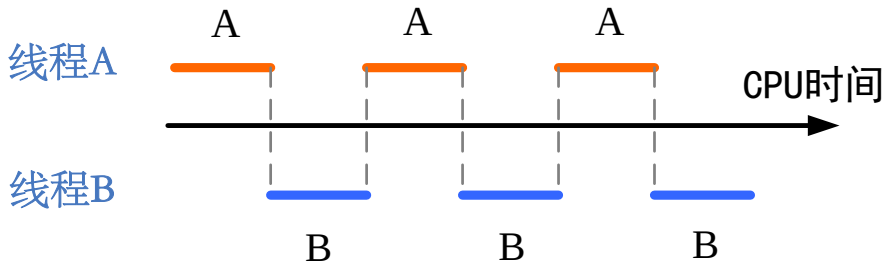
Java多线程的执行

- Java的多线程

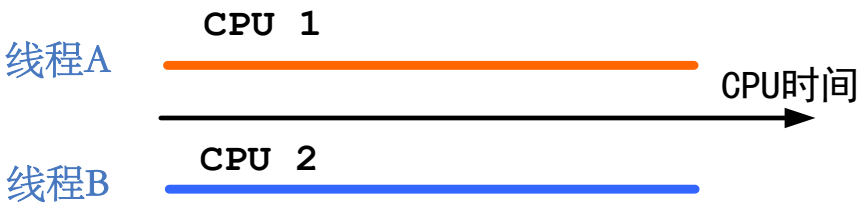
  Java虚拟机获得的总CPU时间内，在若干个独立的可控制的线程之间切换。

- 单CPU如何实现多线程的并行执行
  - 划分极短的CPU时间片
  - 各线程轮流占用一个时间片

并发



并行



Java程序的执行

- 每个Java程序都有一个缺省的主线程
  - Java应用程序总是从主类的main()方法开始执行
  - Main()方法执行过程中可以创建其它线程
- 程序执行情形
  - 如果main()方法中没有创建其它的线程，main()方法返回时，结束Java应用程序
  - 如果main()方法中创建了其它线程，在主线程和其它线程之间轮流切换执行，保证每个线程都有机会使用CPU
  - 直到程序中所有线程都结束，Java应用程序才结束

线程的状态

- 线程可以处于以下五种状态之一：
  - 新 建(New)：
    - new关键字创建后
    - 分配了内存，初始化了其成员变量的值

当线程处于新建状态时，可以通过Thread类的方法设置线程的属性，如线程的优先级(setPriority)、线程名(setName)和线程类型(setDaemon)等

守护(Daemon)线程是特殊的线程，一般用于在后台为其他线程提供服务。

Java中，isDaemon()：判断一个线程是否为守护线程.

Java中，setDaemon()：设置一个线程为守护线程.

线程的状态

- **就　绪(Runnable)**：
  - start()方法；线程已经启动，等待获得CPU时间片执行，处于可运行状态；如果线程处于运行状态，执行yield()方法后，该线程会被暂时剥夺CPU资源，重新进入就绪状态。
- **运　行(Running)**：
  - 获得CPU时间片的线程开始执行
  - 系统真正执行线程的**run()**方法
  - 可以通过Thread类的**isAlive()**方法判断线程是否处于就绪/运行状态，当线程处于就绪/运行状态时，isAlive返回true；当返回false时，线程可能处于阻塞状态，或停止状态。

线程的状态

- **阻　塞(Blocked)**：
  - 一个正在运行的线程因某些原因不能继续运行时，就进入阻塞状态
  - 一个线程进入**阻塞状态**(暂停执行)，可能有如下**原因**：
    1. 通过调用sleep(milliseconds)使线程进入休眠状态，在这种情况下，线程在指定的时间内不会运行。
    2. 通过调用wait( )使线程挂起。直到线程得到了notify( )或notifyAll( )消息，线程才会进入就绪状态
    3. 线程在等待某个输入/输出完成。
    4. 线程试图在某个对象上调用其同步控制方法，但是对象锁不可用。

线程的状态

- 如：当多个线程试图进入某个同步区域(Synchronized)时，没能进入该同步区域的线程会被置入锁定集(Lock Pool)，直到获得该同步区域的锁，进入就绪状态。

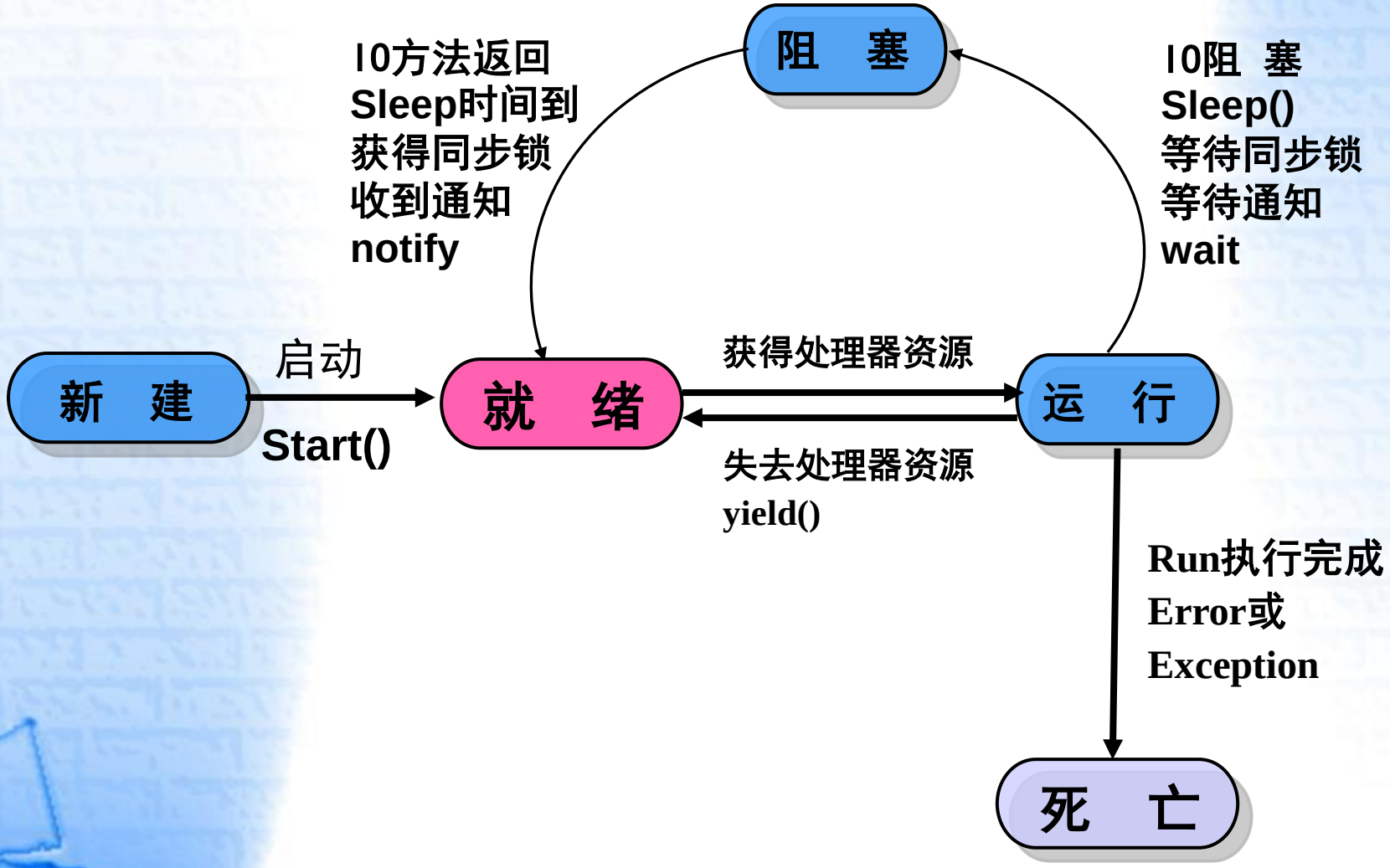
❖ **死　亡(Dead)**：

- 线程在run()方法执行结束后会进入死亡状态。如果线程执行了interrupt()或stop()方法，那么它也会以异常退出的方式进入死亡状态。

线程的状态

- 终止线程的三种方法
  - 使用退出标志，使线程正常退出，也就是当run方法完成后线程终止，推荐使用。
  - 使用stop方法强行终止线程（这个方法不推荐使用，因为stop和suspend、resume一样，也可能发生不可预料的结果）
  - 使用interrupt方法中断线程

线程状态的转换


阻　塞
IO方法返回
Sleep时间到
获得同步锁
收到通知
notify
IO阻 塞
Sleep()
等待同步锁
等待通知
wait
新　建
启动
Start()
就　绪
获得处理器资源
运　行
失去处理器资源
yield()
Run执行完成
Error或
Exception
死　亡

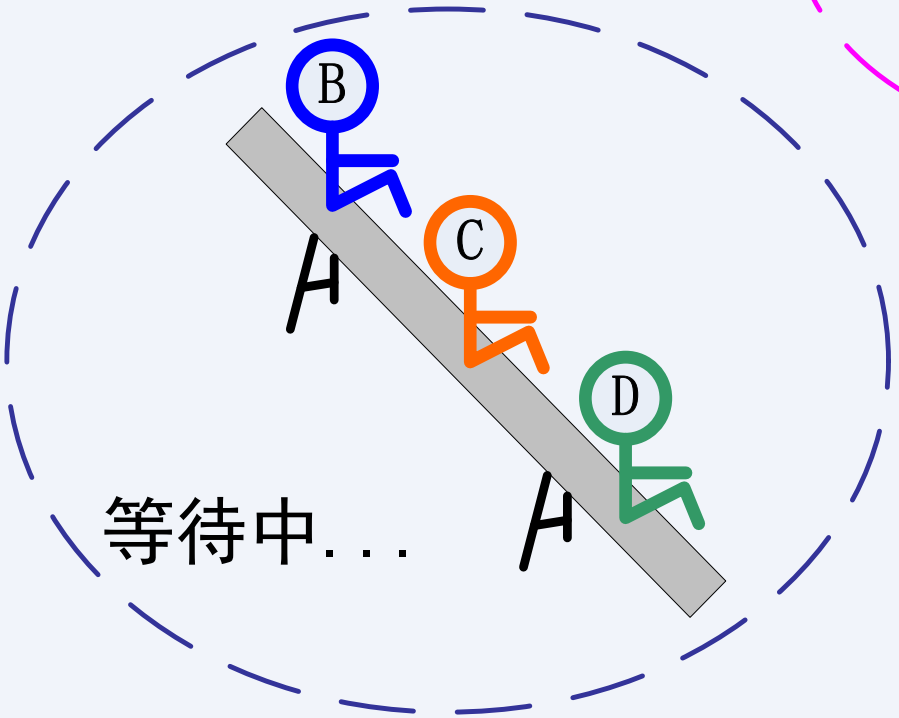
多线程竞争CPU

某寝室……

大家凑钱买了一台

电脑，约定轮流使用





如何“轮流”使用？

1. 等某人“良心发现”，主动放弃
2. 约定好每人每次用时(时间片?)
3. “大股东”，优先使用
4. 某人有急事，临时”抢用”

目　录

1 线 程 概 述

2 Java线程的状态

3 Java线程的创建

4 多线程的同步和通信

Thread类

- 专门用来创建线程和对线程进行操作的类；定义了许多对线程进行操作的方法
- 常用方法
  - final String **getName**(), 返回线程的名称

    默认情况下，主线程的名字为main，用户启动的多条线程的名称依次为Thread-0、Thread-1…等
  - final void **setName**(String name), 设置线程名称
  - static Thread **currentThread**(), 获取当前线程
  - void **start**(), 启动线程
  - void **run**(), 线程运行
  - final boolean **isAlive**(),

    判断某条线程是否已经死亡

    新建、死亡状态返回false

详细内容见软件包 java.lang的Thread类

创建线程方法一

- 编写自己的类
  - 继承java.lang.Thread类
  - 该子类应 重写 Thread 类的 run() 方法

    准备在线程中完成的工作放在run() 方法实现
- 写法之例

```
1. class MyThread extends Thread { // 定义MyThread类
2.     //定义成员属性……
3.     MyThread ( ) { // 构造方法
4.         ……
5.     }
6.     public void run( ) {
7.         // 完成要完成的工作
8.             ……
9.     } }
```

示 例

- 在main方法中创建一个循环，当循环到20遍的时候，创建2个线程。观察main线程和这2个线程的运行情况，并打印输出当前正在运行的线程和运行次数（每个线程循环10遍以打印当前正在运行的线程及次数）。

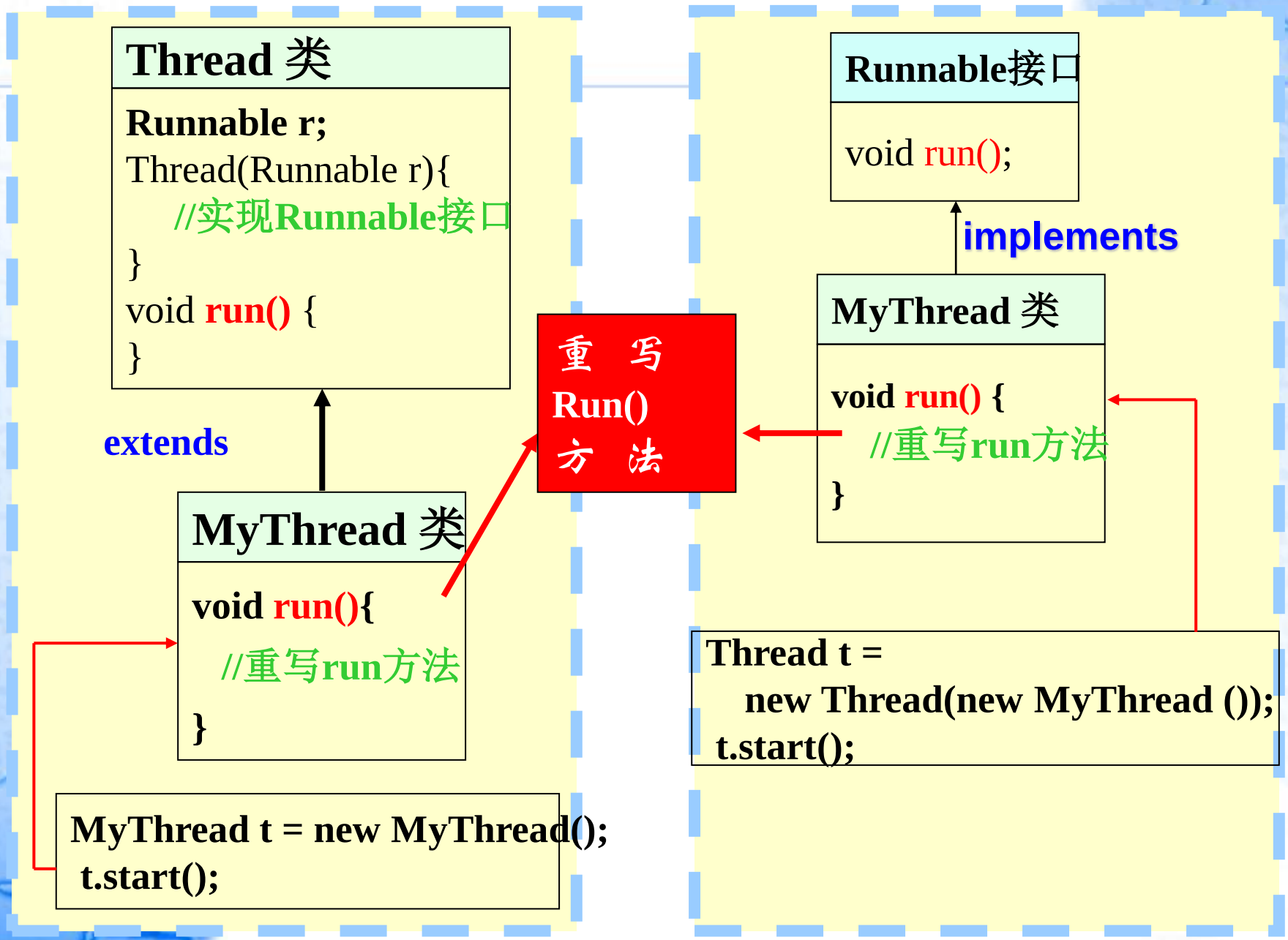
ThreadTest.java

创建线程方法二

- 编写自己的类
  - 实现 Runnable 接口
  - 该子类应重写 Runnable 接口的 run() 方法

    准备在线程中完成的工作放在run() 方法实现
- 写法之例：

```
1. class MyThread  implements  Runnable {
2.     //定义成员属性
3.         ……
4.     MyThread ( ) { // 构造方法
5.         ……  }
6.     public void run( ) {
7.         // 完成要完成的工作
8.          ……
9.     } }
```


Thread 类
Runnable r;
Thread(Runnable r){
//实现Runnable接口
}
void run() {
}
extends
MyThread 类
void run(){
//重写run方法
}
MyThread t = new MyThread();
t.start();
重 写
Run()
方 法
Runnable接口
void run();
implements
MyThread 类
void run() {
//重写run方法
}
Thread t =
new Thread(new MyThread ());
t.start();

示 例

- 修改ThreadTest.java中线程类，使其实现Runnable接口，并完成相同的功能

ThreadTest.java

多线程程序设计. 练习

- 程序功能需求
  - 程序完成两个子任务
  - 每个子任务各自在命令行打印信息
  - 打印内容为：
    - 自己的名称和遍数
    - 打印结束提示
- 解决方法
  - 设计自己的线程类

    （派生自Thread类 或 实现Runnable接口）
  - 在主线程（main方法）中创建两个线程

Thread_1.java

```
//通过继承Thread类实现多线程
class myThread extends Thread {
    private String tname;   //线程名
    private int count;      //循环次数
    //构造方法
    myThread (String s, int c){
        tname = s;
        count = c;
    }
  //一定要实现run方法
   public void run() {
      for (int i=1; i<=count; i++){
       System.out.println  (tname + i));
      }
         System.out.println
          (tname + " finished !");
    }
}
```

```
 //主类
public class Thread_1{
     public static void main(String
args[]){
    //创建线程A
      myThread ta =
              new myThread("A", 10);
     //创建线程B
      myThread tb =
              new myThread("B", 10);
     //  调用start方法
      ta.start();
      tb.start();
   }
}
```

1. 先创建myThread的两个对象ta和tb
2. 分别调用其start方法启动两个线程

目　录

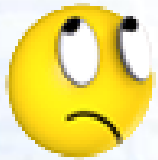
1 线 程 概 述

2 Java线程的状态

3 Java线程的创建

4 多线程的同步和通信

线程并发引起的不确定性



想要实现，两个储户从一个存款帐户中取钱，保证余额正确。

```
class Cbank{   //银行类
    private static int s=10000;
    public static void sub(int m){  //取款
        int temp=s;
        temp=temp-m;
        try{
          Thread.sleep((int)(1000*Math.random()));
        }
        catch(InterruptedException e){ }
        s=temp;
        System.out.println("s="+s);
    }
}
class Customer extends Thread{  //储户类
    public void run(){
        for(int i=1; i<=5; i++)
        Cbank.sub(100);
    }
```

```
class Thread_3  {
      public static void main(String args[])
      {
            Customer Customer1=new Customer( );
            Customer Customer2=new Customer( );
            Customer1.start();
            Customer2.start();
      }
}
```

会出乱子么？



Thread_3
.java

多线程间的关系

- 在多线程环境中，可能会有两个甚至更多的线程试图同时访问一个有限的资源（临界资源，在同一时刻只能有一个线程访问的资源）。必须对这种潜在资源冲突进行预防
  - 同步
- 解决方法：
  - 在线程使用一个资源时为其加锁即可。
  - 访问资源的第一个线程为其加上锁以后，其他线程便不能再使用那个资源，除非被解锁。

线程同步

- 为什么需要同步
  - 线程同步是为了防止多个线程访问同一个数据对象时，对数据造成破坏
  - 线程的同步是保证多线程安全访问竞争资源的一种手段
- 同步和锁
  - Java中每个对象都有一个内置锁
  - 获得一个对象的锁也称为获得锁、锁定对象、在对象上锁定，或在对象上**同步**。当程序运行到**synchronized**同步方法或代码块时该对象锁才起作用。
  - 一个对象只有一个锁。所以，如果一个线程获得该锁，就没有其他进程可以获得锁，直到第一个线程释放锁。这也意味着任何其他线程都不能进入synchronized方法或代码块，直到该锁被释放。释放锁是指持锁进程退出了synchronized同步方法或代码块

Java中的线程同步

- Java中实现线程同步，需要完成两个操作
  - 把竞争访问的资源标识为private
  - 同步访问资源的代码，使用synchronized修饰方法或代码块，例如：synchronized void f() { /* ... */ }。当synchronized方法执行完成或发生异常时，会自动释放锁。
- 当一个线程A使用一个synchronized修饰的方法时，其它线程想使用这个方法时就必须等待，直到线程A 使用完该方法 （除非线程A主动让出CPU资源）
- 如果想保护某些资源不被多个线程同时访问，可以强制通过synchronized方法访问那些资源。
  - 调用synchronized方法时，对象就会**被锁定**
  - 当synchronized方法执行完或发生异常时，会自动释放锁
  - 被synchronized保护的数据应该是私有（private）

线程互斥之例

```
class Cbank  {
    private static int s=1000;
    public synchronized static void sub(int m){
      int temp=s;
      temp=temp-m;
      try  {
       Thread.sleep((int)(1000*Math.random()));
      }
     catch(InterruptedException e)
        { }
     s=temp;
     System.out.println("s="+s);
    }
}
```

```
class Customer extends Thread   {
    public void run()  {
        for(int i=1; i<=5; i++)
        Cbank.sub(100);
    }
 }
//main class
class Thread_4 {
   public static void main(String args[]){
   Customer Customer1=
                        new Customer( );
   Customer Customer2=
                        new Customer( );
   Customer1.start();
   Customer2.start();
  }
}
```

wait()、notify 和notifyAll()方法

- **挂起** 让线程暂时让出CPU的使用权限，暂停执行
  - 挂起一个线程需使用**wait**方法，即让准备挂起的线程调用wait 方法，主动让出CPU的使用权
  - Java 2中已经废止了以前的**suspend( )**方法
- **通知** 恢复处于挂起状态线程的运行
  - 其它线程在占有CPU资源期间，让挂起的线程的目标对象执行**notify/notifyAll**方法，使得挂起的线程继续执行
  - **notify**可以唤醒等待队列中**第一个**等待同一共享资源的线程，并使该线程退出等待队列，进入可运行状态
  - **notifyAll**可以使所有正在等待队列中同一共享资源的线程从等待状态退出，进入可运行状态，此时，优先级最高的线程最先执行。

wait(), yield(), sleep()

- 都是中止当前线程线程的执行
- 区别
  - **wait()**：“挂起”
    线程将释放所有资源，等得到通知后在参加资源竞争；启动办法是notify() 和notifyAll()方法
  - **yield()**: “让步”
    线程放弃当前的CPU使用权，但将CPU资源让出来后马上重新参加CPU资源竞争；线程会自动回到执行状态。
  - **sleep()**: “休眠”
    使线程停止执行一段时间，该时间由你给定的毫秒数决定；线程会自动回到执行状态

wait(), yield(), sleep()

- 具体调用方法上
  - **sleep()**和**yield()**是Thread类定义的**静态方法**，使用时由Thread类直接调用，如：Thread.sleep(1000)、Thread.yield()
  - **wait()**方法需要由线程正在访问的java对象调用，如：

```
public class Test  {
    object obj;
    public void foo()  {
        synchronize(obj)  {
            while(<condition does not  hold>)  {
                obj.wait()
            }
```

  - **wait()**是java.lang.Object定义的方法，是**final**类型的。

Wait方法示例

TestWait.java

- “->”是Java 8新增的Lambda表达式，变量和临时代码块的分隔符，即：

  (变量)->{代码块}

  核心原则“可推导，可省略”→如果代码块只有一个表达式，大括号可以省略。如果变量类型可以自动推断出来，可以不写变量类型。
- 它可以对匿名内部类的写法进行简化。
- 它是函数式编程思想的一个体现。让我们不用去关注是什么对象，而是更加关注我们对数据进行了什么操作。

->对匿名内部类的简化

- 如果匿名内部类是一个接口的类，并且只有一个抽象方法需要重写

```
new Thread(new Runnable(){
            public void run()
            {
               System.out.println ("Constant dripping wears away a stone.");
            }
            }).start();
```

```
new Thread(()->{

        System.out.println ("Constant dripping wears away a stone.");

    }).start();
```

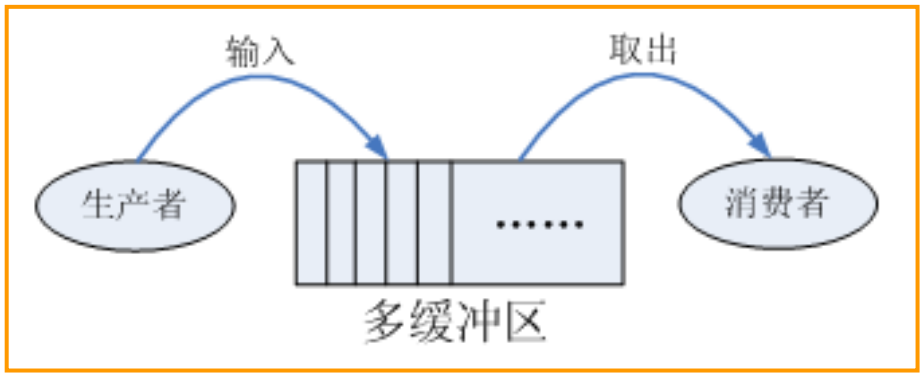
Interrupt()

- interrupt方法常用来吵醒休眠的线程
- 当一些线程调用sleep()方法处于休眠状态时，一个占有CPU资源的线程可以让休眠的线程调用interrupt方法吵醒自己，即导致休眠的线程发生InterruptedException异常，从而结束休眠，重新排队等待CPU资源。
- 例：
  - 2个线程student和teacher，其中，student准备睡一个小时后再开始上课，teacher在输出3句“上课”后吵醒休眠的线程student。

TestInterrupt.java

经典同步 - 生产者消费者问题

- 设生产者和消费者共用n个缓冲区。生产者负责生产产品送到缓冲区，消费者取走缓冲区中的产品。
  - 设每缓冲区的大小只能装入一个产品的信息，
  - 生产者在完成将产品送入缓冲区后，必须等消费者将产品取出后才能放入下一个产品
  - 只有在生产者将产品送入缓冲区后消费者才能读取产品
- 生产者和消费者存在直接制约关系，必须协调同步。



生产者消费者问题之例

- ProducerConsumer
  - ProducerConsumerTest.java
  - Buffer.java

    缓冲区类
  - Producer.java

    生产者类
  - Consumer.java

    消费者类

Code

线程的联合

- 一个线程A在占有CPU资源期间，可以让其他线程调用join()和本线程联合，如：B.join()；称A在运行期间联合了B。
- 如果线程A在占有CPU期间联合了B，那么A线程将立刻中断执行，一直等到它联合的线程B执行完毕，A线程再重新排队等待CPU资源，以便恢复执行。如果A准备联合的B线程已经结束，那么B.join()不会产生任何效果。
- 示例
  - 有汽车制造商和顾客2个线程，如果先启动的是顾客，则顾客需要等汽车制作商制造好汽车后，才能够进行购买。请用线程的思想进行描述。

TestJoin.java

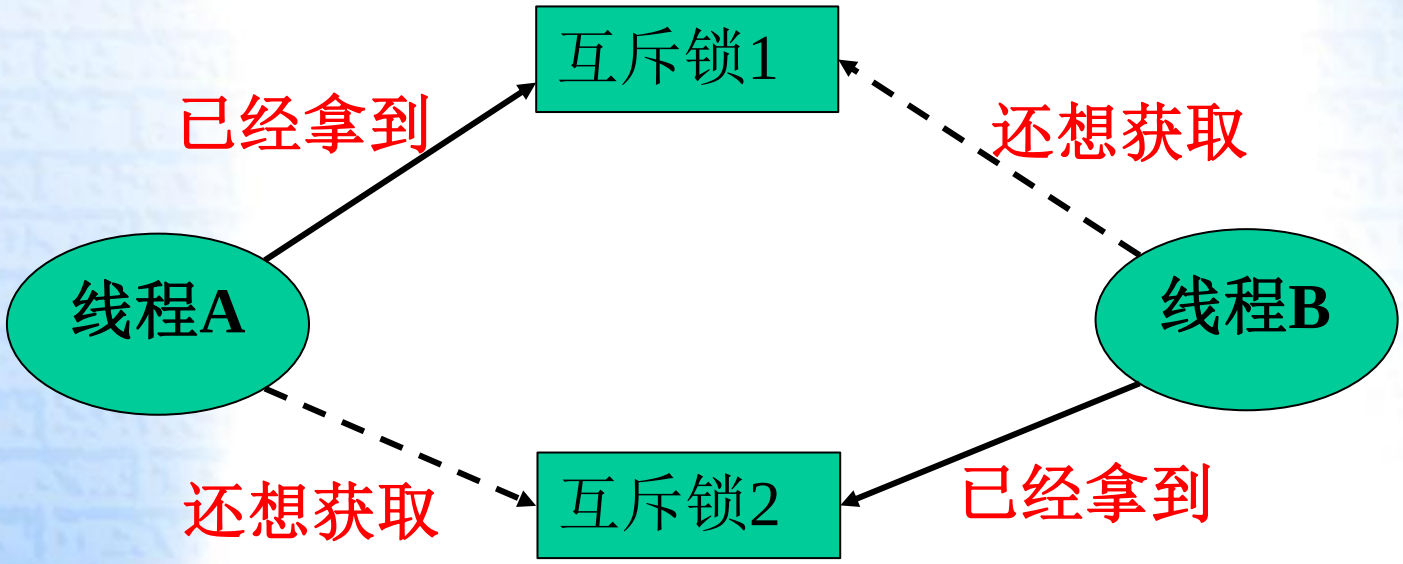
```java
class DataOutput extends Thread
{
    public void run( )
    {
        int i=10;
        while(i<=15) System.out.println((i++));
    }
}
```

```java
class TestJoin2
{
    public static void main(String args[ ]) throws InterruptedException
    {
        for(int i=0; i<100; i++)
        {
            System.out.println (Thread.currentThread().getName() +"循环次数" +  i);
            if(i == 20)
            {
                Thread th=new DataOutput ( );
                th.start( );
                th.join( ); 此行代码，要与不要，结果分别有什么区别？
            }
        }
    }
}
```

TestJoin2.java

死锁（Deadlock）

- Java中的死锁是指多个线程同时占用一些共享资源且彼此相互等待，从而导致所有的线程都被阻塞，不能继续执行程序的情况。



```java
public class TestLock
{
   // 定义2个锁定的对象
   private Object lock1 = new Object();
   private Object lock2 = new Object();

   public void method1()
   {
      // 锁定对象1
      synchronized (lock1)
      {
         System.out.println("Method 1: 获取对象lock1的锁");
         try {
            Thread.sleep(100);
         } catch (InterruptedException e) {
            e.printStackTrace(); }

         // 锁定对象2
         synchronized (lock2)
         {
            System.out.println("Method 1: 获取对象l
         }
      }
   }
```

```java
public void method2()
{
   // 锁定对象2
   synchronized (lock2)
   {
      System.out.println("Method 2: 获取对象lock2的锁");
      try {
         Thread.sleep(100);
      } catch (InterruptedException e) {
         e.printStackTrace();     }

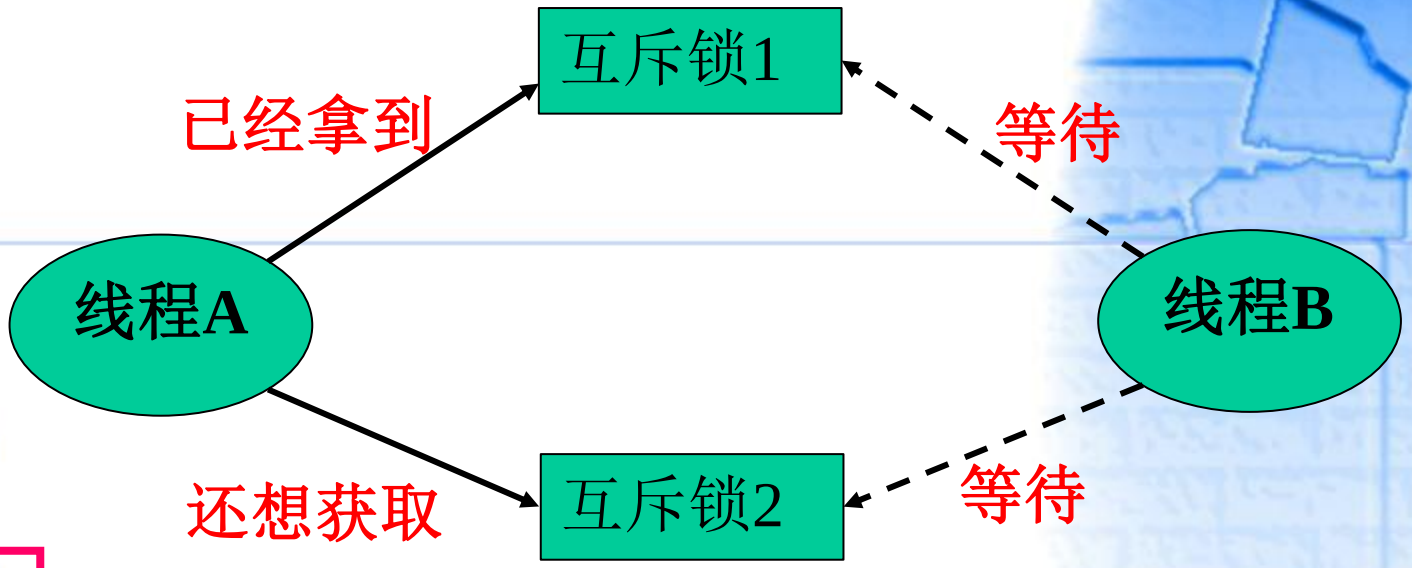
      // 锁定对象1
      synchronized (lock1)
      {
         System.out.println("Method 2: 获取对象lock1的锁");
      }
   }
}
```

```
public static void main(String[] args)
{
    final TestLock example = new TestLock();

    Thread thread1 = new Thread(new Runnable()
    {
        public void run()
        {
            example.method1();
        }
    });

    Thread thread2 = new Thread(new Runnable()
    {
        public void run() {
            example.method2();
        }
    });

    //开启线程
    thread1.start();
    thread2.start();
}
```



修改

```
public void method2()
{
   // 锁定对象2
   synchronized (lock1)
   {
      System.out.println("Method 2: 获取对象lock1的锁");
      try {
         Thread.sleep(100);
      } catch (InterruptedException e) {
         e.printStackTrace();      }

      // 锁定对象1
      synchronized (lock2)
      {
         System.out.println("Method 2: 获取对象lock2的锁");
      }
   }
}
```

TestLock.java

本章总结

- 本章学习Java多线程的基本思想、方法。
  - 线程概念的理解
  - 线程的状态
  - Java程序中创建线程的方法
  - 线程常用方法
  - 线程的同步与互斥

作 业

1. 思考题（不交）
   1. 何为线程和多线程？如何建立多线程？
   2. 线程的生命周期由哪几种状态？各状态之间分别用哪些方法切换？
   3. java程序实现多线程有哪些途径？
2. 编写两个线程（要交）：
   - 第一个线程计算2-1000000之间的质数及个数
   - 第二个线程计算1000000-2000000之间的质数及个数

Class is Dismissed!

